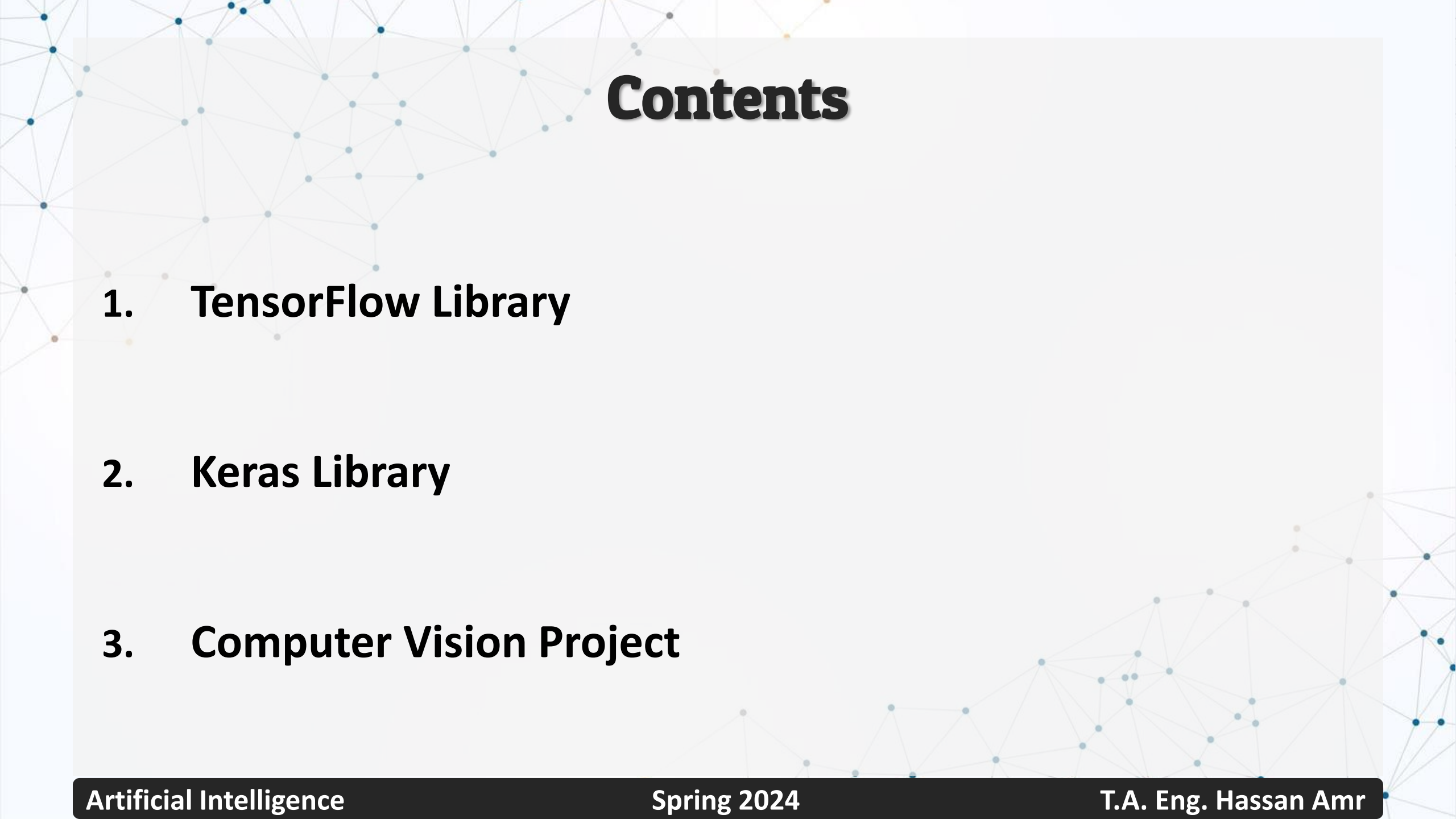


Artificial Intelligence (Spring 2024)

AI Week 4

[TensorFlow Library - Keras Library - Computer Vision Project]



Contents

- 1. TensorFlow Library**
- 2. Keras Library**
- 3. Computer Vision Project**

TensorFlow Library

- TensorFlow is a **free and open-source software library** for machine learning and artificial intelligence.
- It can be used across a range of tasks but has a particular **focus on training and inference of deep neural networks**.
- It was developed by the Google Brain team for Google's internal use in research and production.

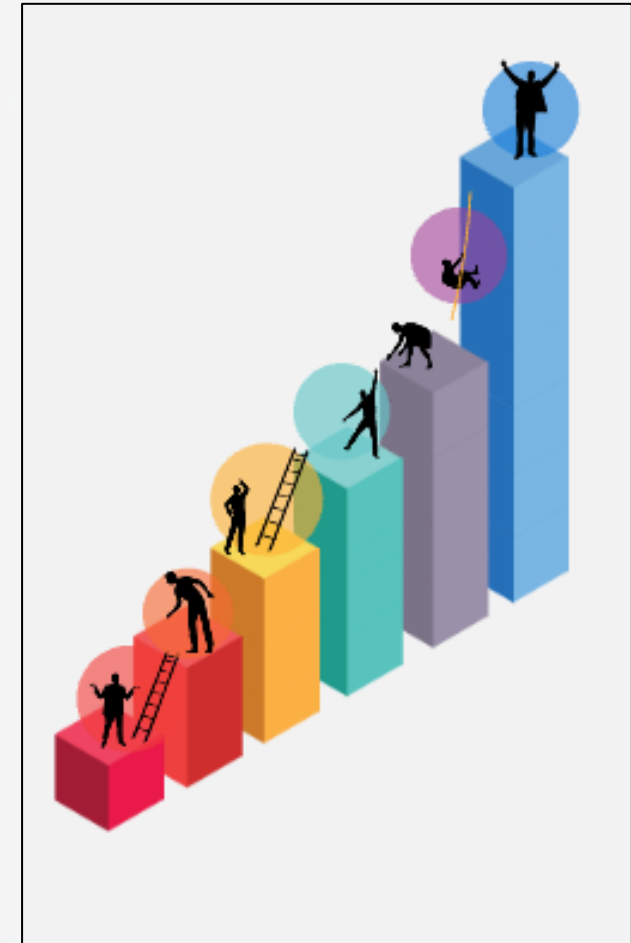


TensorFlow

TensorFlow Library Advantages

➤ Why TensorFlow?

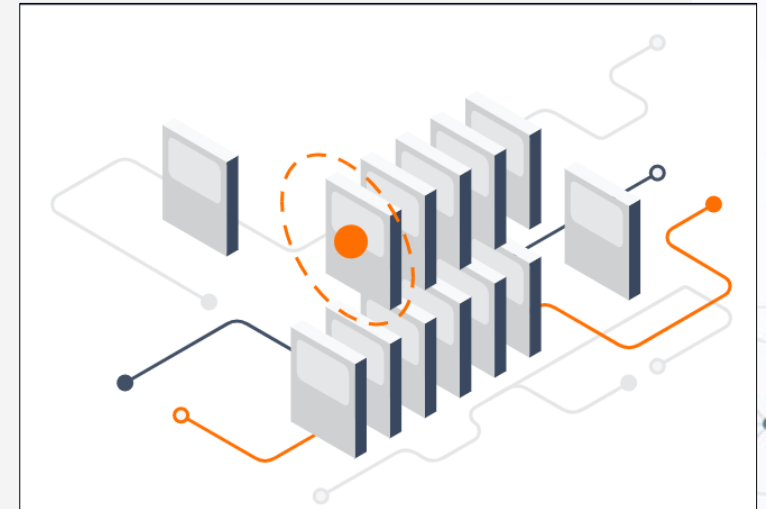
- ✓ Whether you're an **expert or a beginner**, TensorFlow is an end-to-end platform that makes it **easy** for you to **build and deploy ML models**.



TensorFlow Library Advantages

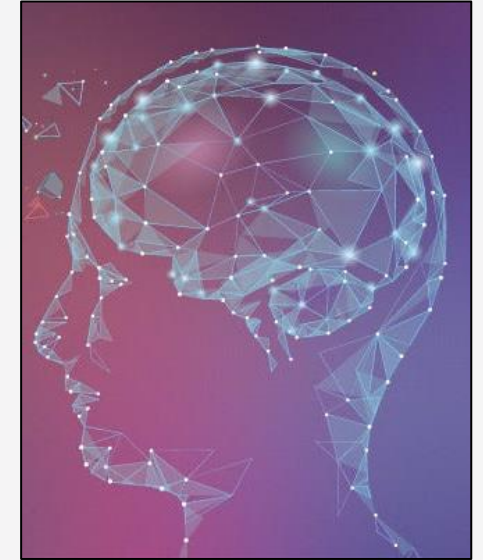
➤ Easy Model Building

- ✓ TensorFlow offers **multiple levels of abstraction** so you can choose the right one for your needs. Build and train models by using the **high-level Keras API**, which makes getting started with TensorFlow and machine **learning easy**.
- ✓ If you need more flexibility, eager execution allows for immediate iteration and intuitive debugging. For **large ML training tasks**, use the **Distribution Strategy API** for **distributed training on different hardware configurations** without changing the model definition.



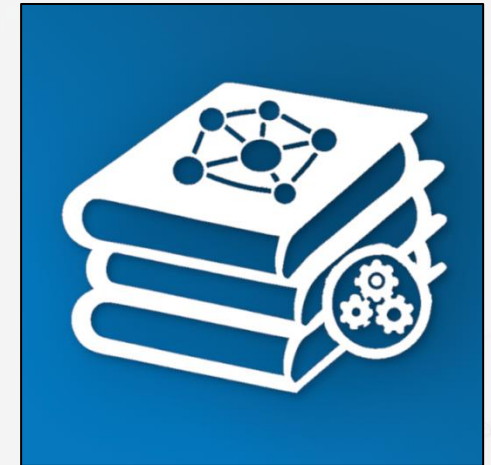
TensorFlow Library Advantages

- **Robust ML production anywhere**
- ✓ TensorFlow offers **multiple levels of abstraction** so you can choose the right one for your needs. Build and train models by using the **high-level Keras API**, which makes getting started with TensorFlow and machine **learning easy**.
- ✓ If you need more flexibility, eager execution allows for immediate iteration and intuitive debugging. For **large ML training tasks**, use the **Distribution Strategy API** for **distributed training on different hardware configurations** without changing the model definition.



TensorFlow Library Advantages

- **Powerful experimentation for research**
- ✓ Build and train **state-of-the-art models** without sacrificing **speed or performance**.
- ✓ TensorFlow gives you the **flexibility and control with features** like the Keras Functional API and **Model Subclassing API** for creation of complex topologies. For **easy prototyping and fast debugging**, use eager execution.
- ✓ TensorFlow also supports an ecosystem of powerful add-on libraries and models to experiment with, including **Ragged Tensors**, TensorFlow Probability, **Tensor2Tensor** and **BERT**.



Keras Library

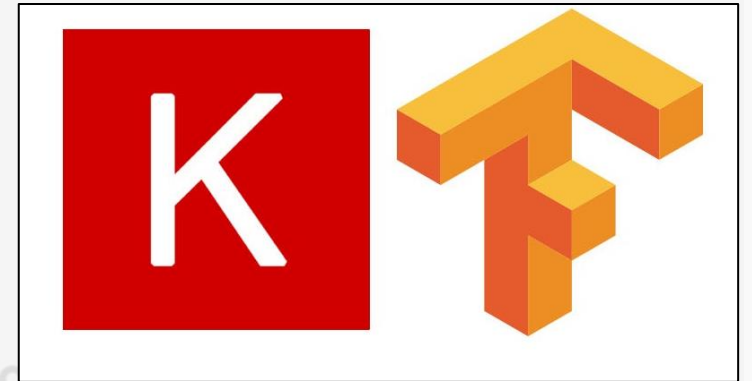
- Keras is an **open-source library** that provides a Python interface for artificial neural networks.
- Keras acts as an **interface for the TensorFlow** library.
- Keras contains numerous implementations of **commonly used** neural-network building blocks such as **layers**, objectives, **activation functions**, **optimizers**, and a host of tools for working with **image and text data** to simplify programming in deep neural network area.



Keras Library Advantages

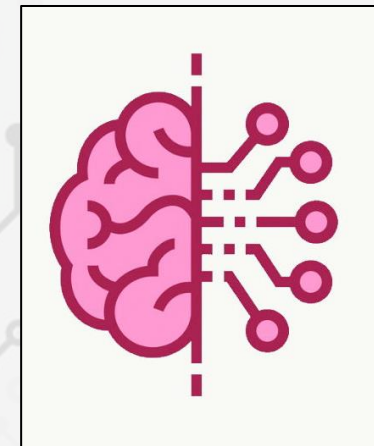
➤ A superpower for developers.

- Keras focuses on debugging **speed**, code elegance & **conciseness**, **maintainability**, and **deployability**.
- When you choose Keras, your **codebase is smaller**, more **readable**, easier to iterate on.
- Your models **run faster** thanks to XLA compilation with JAX and TensorFlow.
- Your models are easier to **deploy across every surface** (server, mobile, browser, embedded) thanks to the serving components from the TensorFlow and PyTorch ecosystems, such as TF Serving, TorchServe, TF Lite, TF.js, and more.



Keras Library Advantages

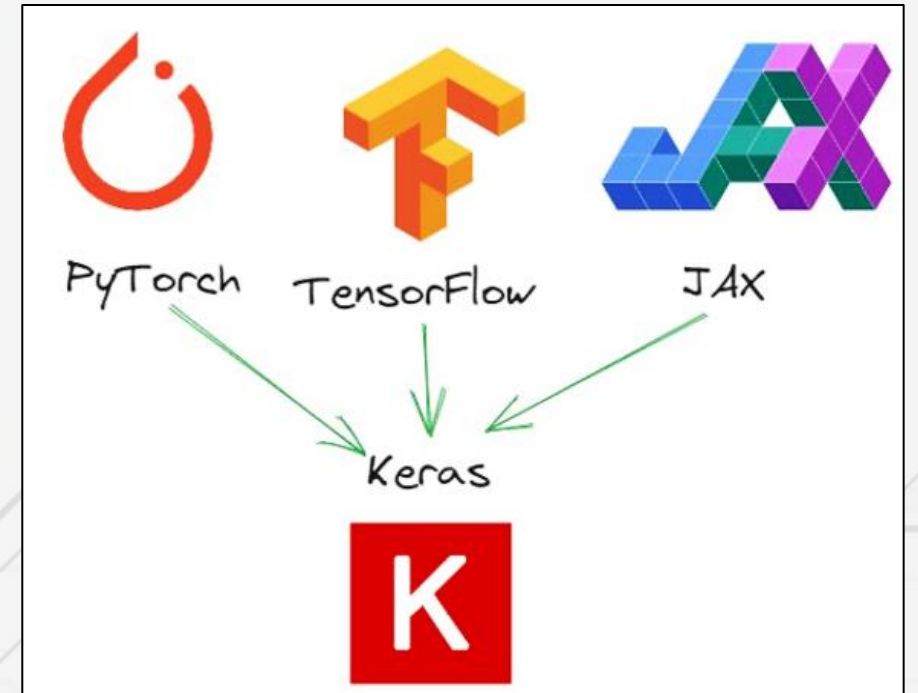
- Deep learning for humans.
- ✓ Keras **follows best practices** for **reducing cognitive load**: it offers **consistent & simple APIs**, it **minimizes the number of user actions** required for common use cases.
- ✓ Keras provides **clear & actionable error messages**.
- ✓ Keras also gives the **highest priority to crafting great documentation** and developer guides.



Keras Library Advantages

➤ Unlock framework optionality.

- ✓ Keras works with JAX, TensorFlow, and PyTorch.
- ✓ It enables you to create **models that can move across framework** boundaries and that can **benefit from the ecosystem of all three** of these frameworks.



Keras Library Advantages

- Exascale machine learning.
- ✓ Keras is an industry-strength framework that can **scale** to **large clusters of GPUs** or an **entire TPU pod**. It's not only possible; it's easy.



Basic Computer Vision Project

- Basic project in computer vision that performs **clothing images classification** for the **MNIST Dataset**, which is one of the largest standard datasets in the world.
- TensorFlow and Keras are always used together and complement each other.
- Create a **neural network** that can classify images in the dataset into **different categories**.

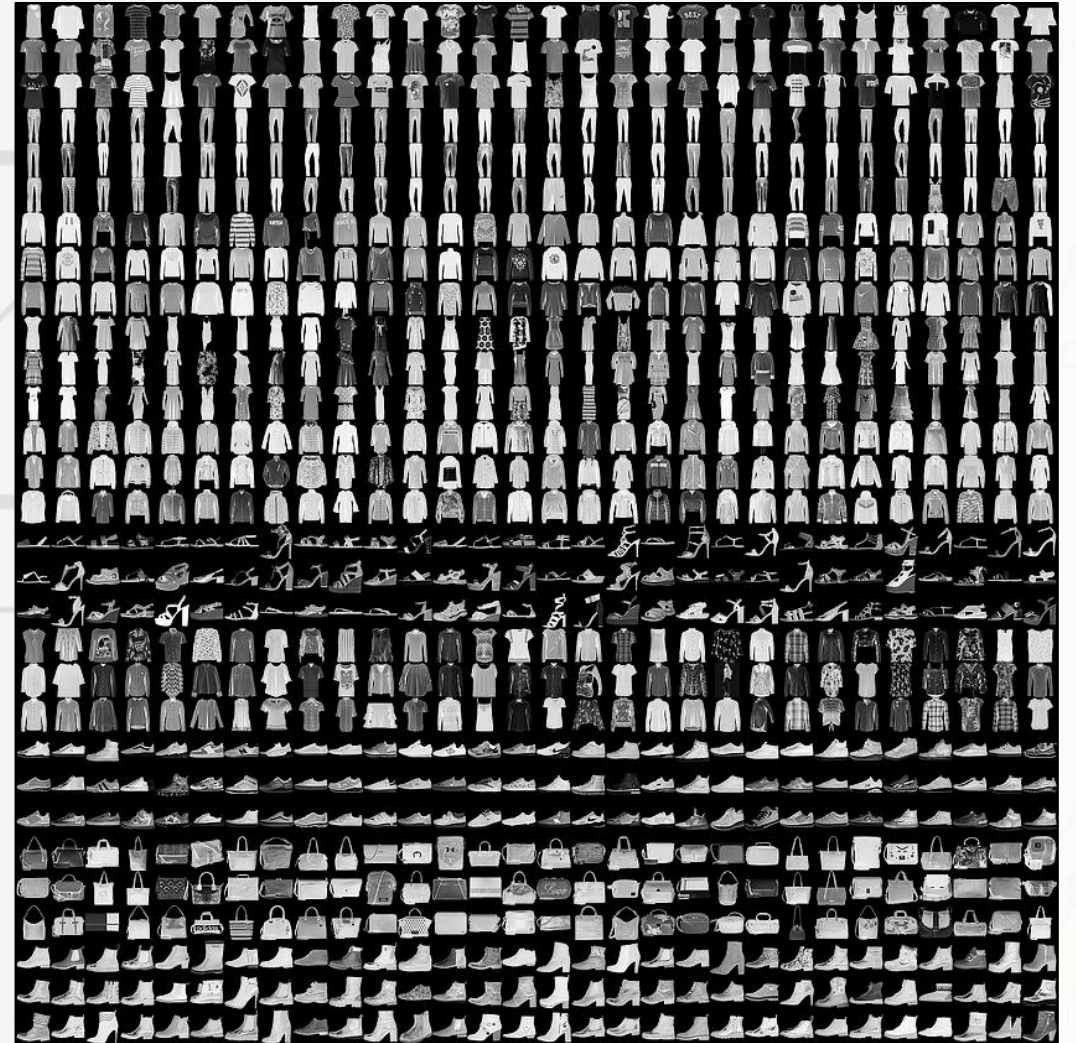


Milestones

1. Import the Fashion MNIST dataset
2. Explore the data
3. Preprocess the data
4. Build the model
5. Set up the layers
6. Compile the model
7. Train the model
8. Feed the model
9. Evaluate accuracy
10. Make predictions

Import the Fashion MNIST dataset

- **Fashion MNIST** (Modified National Institute of Standards and Technology) dataset contains **70,000 grayscale** images in **10 categories**.
- The images show individual articles of clothing at low resolution (**28 by 28 pixels**), as seen here:



Import the Fashion MNIST dataset

```
fashion_mnist = tf.keras.datasets.fashion_mnist

(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()
```

Loading the dataset returns four NumPy arrays:

- The **train_images** and **train_labels** arrays are the training set—the data the model uses to learn.
- The model is tested against the test set, the **test_images**, and **test_labels** arrays.

(60,000 images are used to train the network and 10,000 images to evaluate how accurately the network learned to classify images)

Import the Fashion MNIST dataset

The images are **28x28 NumPy arrays**, with **pixel values ranging from 0 to 255**.

The **labels are an array of integers, ranging from 0 to 9**.

These correspond to the class of clothing the image represents:

Label	Class
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

Import the Fashion MNIST dataset

Each image is mapped to a single label. Since the ***class names are not included with the dataset***, store them here to use later when plotting the images:

```
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',  
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
```



Explore the data

Let's explore the format of the dataset before training the model. The following shows there are **60,000 images** in the **training set**, with each image represented as **28 x 28 pixels**:

```
train_images.shape
```

```
(60000, 28, 28)
```

Explore the data

There are **10,000 images** in the **test set**. Again, each image is represented as **28 x 28 pixels**:

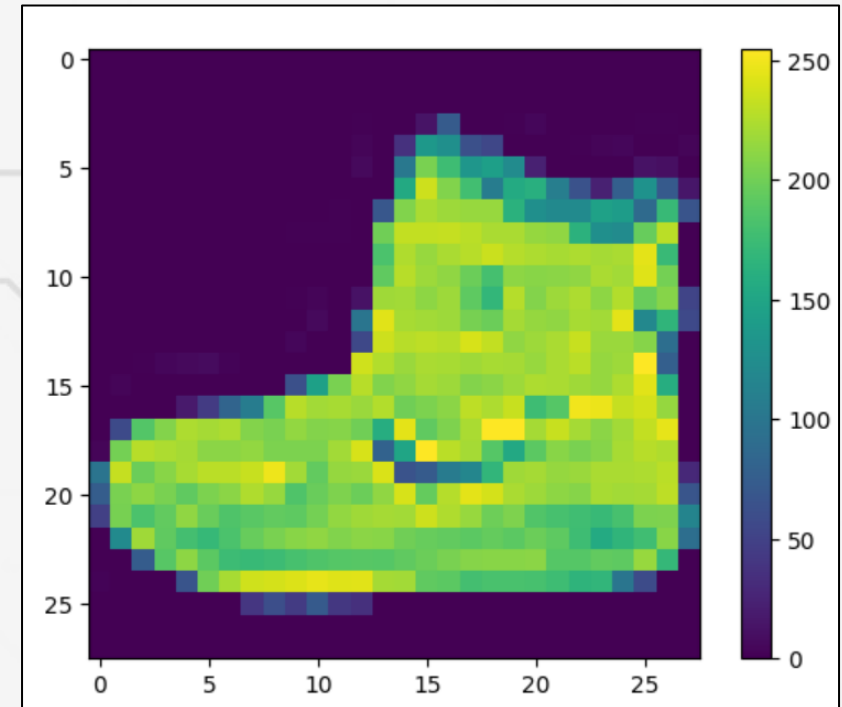
```
test_images.shape
```

```
(10000, 28, 28)
```

Preprocess the data

The data must be preprocessed **before training the network**. If you inspect the first image in the training set, you will see that the pixel values fall in the range of 0 to 255:

```
plt.figure()  
plt.imshow(train_images[0])  
plt.colorbar()  
plt.grid(False)  
plt.show()
```



Preprocess the data

Scale these values to a **range of 0 to 1** before feeding them to the neural network model. To do so, **divide the values by 255**. It's important that the **training set and the testing set be preprocessed in the same way**:

```
train_images = train_images / 255.0
```

```
test_images = test_images / 255.0
```

Preprocess the data

To **verify** that the data is in the correct format and that you're ready to build and train the network, let's **display the first 25 images** from the *training set* and display the class name below each image.

```
plt.figure(figsize=(10,10))
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(train_images[i], cmap=plt.cm.binary)
    plt.xlabel(class_names[train_labels[i]])
plt.show()
```



Build the model

Building the neural network requires **configuring the layers** of the model, then **compiling the model**.

Build the model - Set up the layers

The **basic building block** of a neural network is the **layer**. Layers extract representations from the data fed into them.

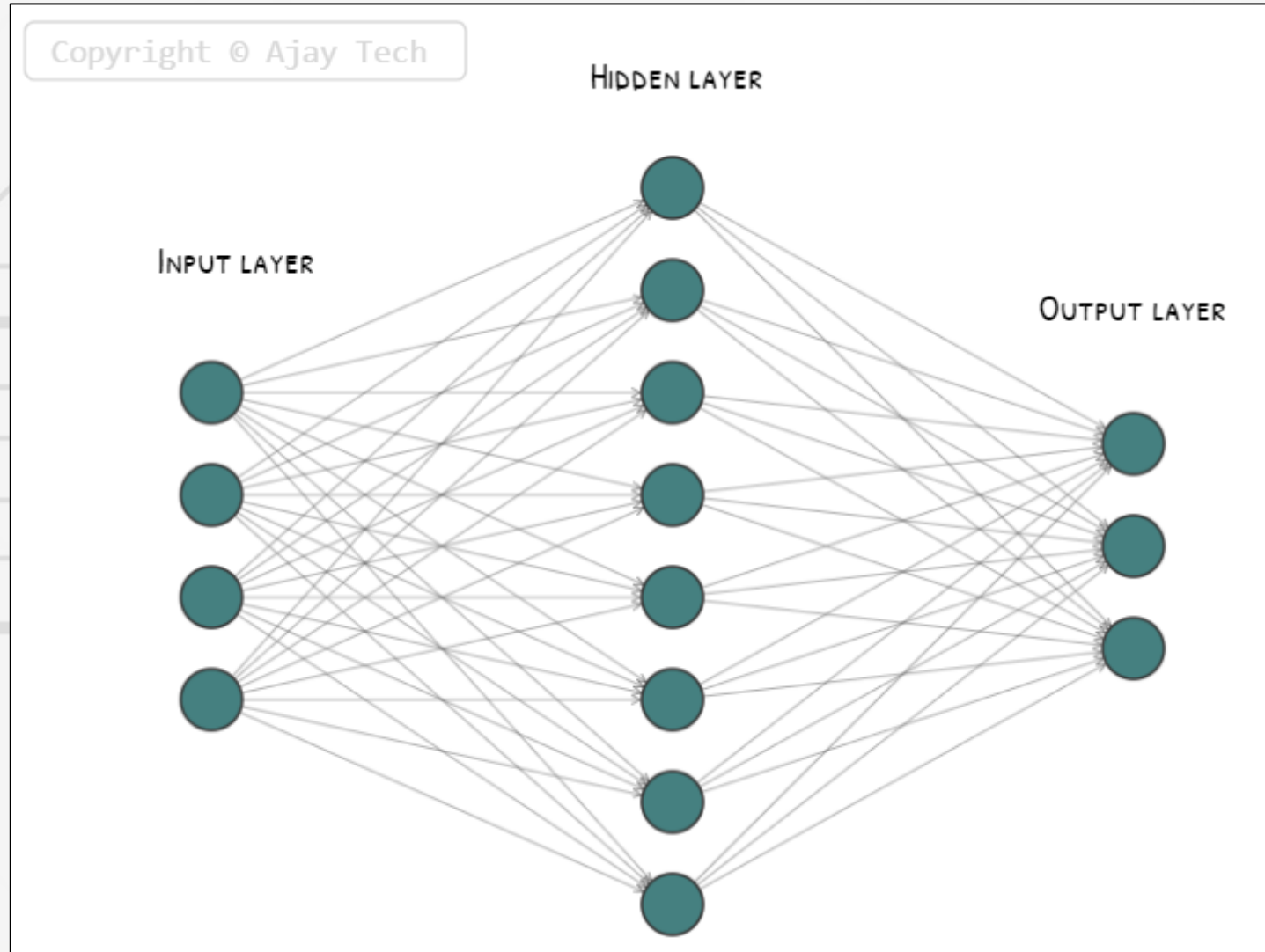
Most of deep learning consists of chaining together simple layers. Most layers, such as `tf.keras.layers.Dense`, have parameters that are learned during training.

```
model = tf.keras.Sequential([  
    tf.keras.layers.Flatten(input_shape=(28, 28)),  
    tf.keras.layers.Dense(128, activation='relu'),  
    tf.keras.layers.Dense(10)  
])
```

Build the model - Set up the layers

- The first layer in this network, `tf.keras.layers.Flatten`, transforms the format of the images from a two-dimensional array (of 28 by 28 pixels) to a one-dimensional array (of $28 * 28 = 784$ pixels). Think of this layer as unstacking rows of pixels in the image and lining them up. This layer has no parameters to learn; it only reformats the data.
- After the pixels are flattened, the network consists of a sequence of two `tf.keras.layers.Dense` layers. These are densely connected, or fully connected, neural layers. The first Dense layer has 128 nodes (or neurons). The second (and last) layer returns a logits array with length of 10. Each node contains a score that indicates the current image belongs to one of the 10 classes.

Build the model - Set up the layers



Build the model - Compile the model

Before the model is ready for training, it needs a few more settings. These are added during the model's compile step:

- **Optimizer** —This is how the model is updated based on the data it sees and its loss function.
- **Loss function** —This measures how accurate the model is during training. You want to minimize this function to "steer" the model in the right direction.
- **Metrics** —Used to monitor the training and testing steps. The following example uses accuracy, the fraction of the images that are correctly classified.

```
model.compile(optimizer='adam',  
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),  
              metrics=['accuracy'])
```



Train the model

Training the neural network model requires the following steps:

1. **Feed the training data to the model.** In this example, the training data is in the `train_images` and `train_labels` arrays.
2. The **model learns to associate images and labels.**
3. You **ask the model to make predictions** about a **test set**—in this example, the `test_images` array.
4. **Verify that the predictions match the labels** from the `test_labels` array.

Train the model - Feed the model

To start training, call the **model.fit** method—so called because it "fits" the model to the training data:

As the model trains, the loss and accuracy metrics are displayed. This model reaches an accuracy of about 0.91 (or 91%) on the training data.

```
model.fit(train_images, train_labels, epochs=10)
```

Epoch 1/10

WARNING: All log messages before absl::InitializeLog() is called are written to STDERR

I0000 00:00:1701918359.788498 80599 device_compiler.h:186] Compiled cluster using XLA! This line is lo

1875/1875 [=====] - 5s 2ms/step - loss: 0.5025 - accuracy: 0.8227

Epoch 2/10

1875/1875 [=====] - 4s 2ms/step - loss: 0.3817 - accuracy: 0.8631

Epoch 3/10

1875/1875 [=====] - 4s 2ms/step - loss: 0.3391 - accuracy: 0.8756

Epoch 4/10

1875/1875 [=====] - 4s 2ms/step - loss: 0.3152 - accuracy: 0.8850

Epoch 5/10

1875/1875 [=====] - 4s 2ms/step - loss: 0.2973 - accuracy: 0.8917

Epoch 6/10

1875/1875 [=====] - 4s 2ms/step - loss: 0.2835 - accuracy: 0.8956

Epoch 7/10

1875/1875 [=====] - 4s 2ms/step - loss: 0.2716 - accuracy: 0.8996

Epoch 8/10

1875/1875 [=====] - 4s 2ms/step - loss: 0.2580 - accuracy: 0.9042

Epoch 9/10

1875/1875 [=====] - 4s 2ms/step - loss: 0.2500 - accuracy: 0.9075

Epoch 10/10

1875/1875 [=====] - 4s 2ms/step - loss: 0.2436 - accuracy: 0.9089

<keras.src.callbacks.History at 0x7fd041332b80>

Train the model - Evaluate accuracy

Next, compare how the model performs on the test dataset:

```
test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2)
print('\nTest accuracy:', test_acc)
```

```
313/313 - 1s - loss: 0.3416 - accuracy: 0.8791 - 597ms/epoch - 2ms/step
```

```
Test accuracy: 0.8791000247001648
```

Train the model - Evaluate accuracy

- It turns out that the accuracy on the test dataset is a little less than the accuracy on the training dataset. This gap between training accuracy and test accuracy represents **overfitting**.
- Overfitting happens when a machine learning **model performs worse on new, previously unseen inputs** than it does on the training data.
- An overfitted model "memorizes" the noise and details in the training dataset to a point where it negatively impacts the performance of the model on the new data.

More about overfitting:

https://www.tensorflow.org/tutorials/keras/overfit_and_underfit#demonstrate_overfitting

https://www.tensorflow.org/tutorials/keras/overfit_and_underfit#strategies_to_prevent_overfitting

Train the model - Make predictions

With the model trained, you can use it to make predictions about some images. Attach a **softmax layer** to **convert the model's linear outputs—logits—to probabilities**, which should be easier to interpret.

```
probability_model = tf.keras.Sequential([model,  
                                         tf.keras.layers.Softmax()])  
  
predictions = probability_model.predict(test_images)
```

Train the model - Make predictions

Here, the model has predicted the label for each image in the testing set. Let's take a look at the first prediction:

```
predictions[0]
```

```
array([2.4707775e-08, 6.4802941e-09, 5.7907998e-09, 2.2175234e-10,  
       2.9547037e-08, 2.6677480e-05, 2.0438068e-07, 2.8663729e-03,  
       4.4412984e-09, 9.9710673e-01], dtype=float32)
```

Train the model - Make predictions

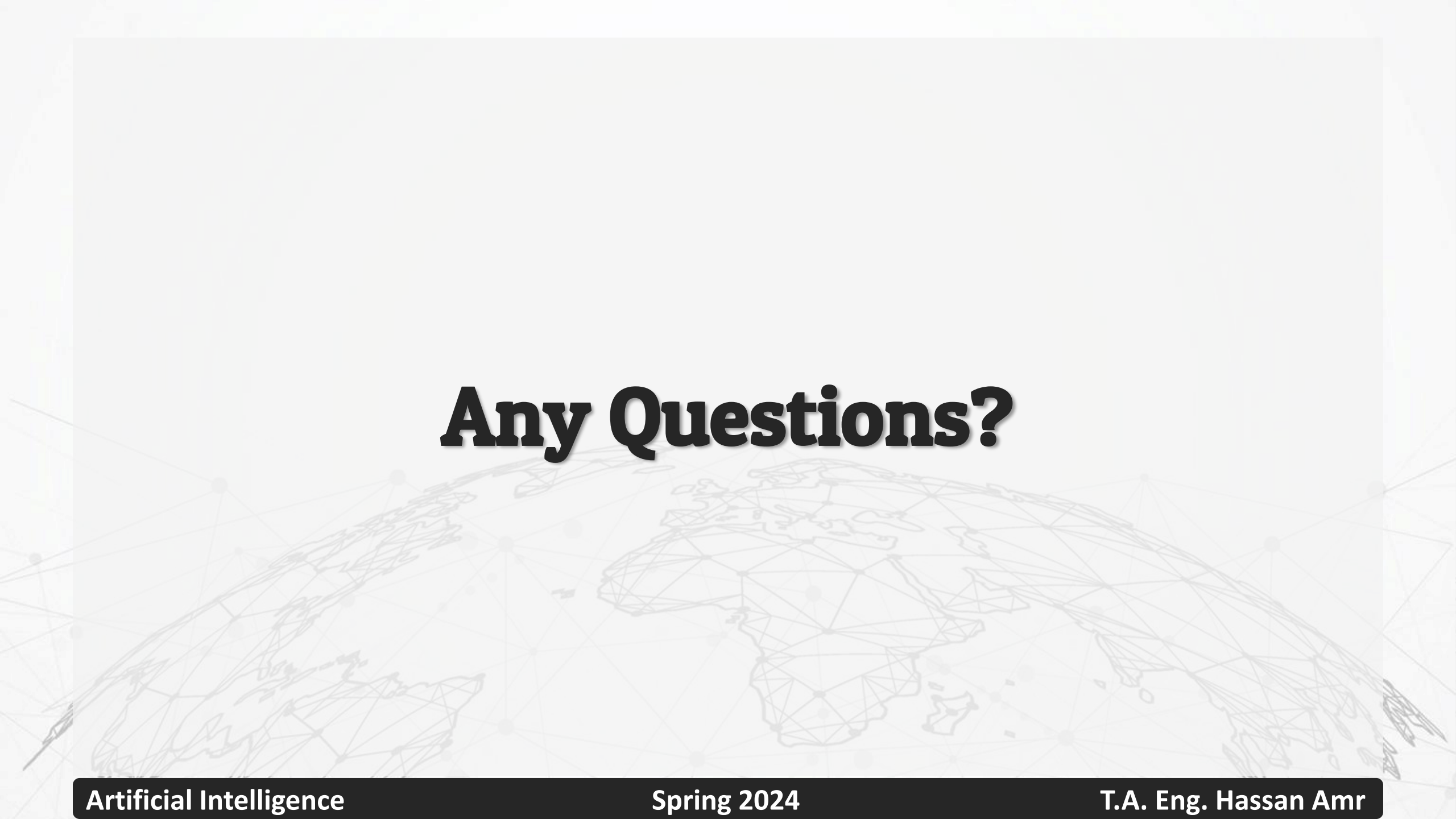
A prediction is an array of 10 numbers. They represent the model's "confidence" that the image corresponds to each of the 10 different articles of clothing. You can see **which label has the highest confidence value:**

```
np.argmax(predictions[0])
```

9

```
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',  
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
```

So, the model is most confident that this image is an ankle boot, or `class_names[9]`

The background features a stylized globe with a network overlay. The globe is composed of a series of interconnected lines and dots, creating a mesh-like structure that covers the entire surface. The lines are thin and grey, while the dots are small and light grey. The overall effect is a modern, technological representation of the Earth.

Any Questions?